

基于 ESP32 智能眼镜的 视觉导航与语音交互系统

AI Glasses for Navigation · Development Log

项目周期: 2026 年 6 月 3 日 — 2026 年 8 月 31 日

文档类型: 项目开发日志 (含方案设计 / 实验测试 / 问题排查 / 版本迭代 / 团队分工)

技术栈: ESP32-S3 · Go · Python · Vue 3 · DashScope/Qwen · 高德地图

文档格式: PDF (≤ 50 MB)

目录

一 项目概述

二 开发时间线总览

三 分阶段开发记录

四 关键问题排查与解决

五 实验与测试数据

六 版本迭代记录

七 团队分工

八 总结与展望

附录A 方案设计与系统架构图（佐证材料）

一、项目概述

1.1 项目背景与目标

视障与低视力人群在独立出行中面临三大核心痛点：**环境感知受限**（难以判断盲道走向、前方障碍物）、**过街判断困难**（斑马线对准、红绿灯状态难以感知）、**交互方式不自然**（需要语音即时的信息获取与问答）。

本项目以低成本可穿戴硬件为切入点，设计并实现一套**基于 ESP32 智能眼镜的视觉导航与语音交互系统**：眼镜端负责音视频采集与语音播报，云端/本地 AI 服务负责视觉理解与自然语言交互，从而在有限算力与网络条件下，为用户提供**盲道导航、过街引导、障碍物避让、寻物引导、语音问答与盲道友好路线规划**等能力。

1.2 系统总体架构

系统采用「四层结构 + 端云协同」设计，各层职责如下：

层级	目录	主要作用	技术框架
设备端	firmware/xiao_sense_ws_stream	摄像头/麦克风采集、扬声器播放、Wi-Fi/UDP 通信	Arduino ESP32、FreeRTOS、ESP32 Camera、I2S/PDM
后端中转层	server	接收设备音视频流、维护设备状态、转发视觉任务、对接 DashScope Realtime	Go、Gin、Gorilla WebSocket

AI/视觉 Worker	python_worker	盲道导航、斑马线/红绿灯检测、障碍物检测、寻物引导、ASR 与多模态问答、路线规划	Python、FastAPI、OpenCV、PyTorch、Ultralytics、MediaPipe、DashScope
前端展示层	esp32-glass-front	实时画面预览、设备状态、AI 状态、导航控制	Vue 3、Vite、Ant Design Vue

核心数据流：ESP32 采集摄像头 JPEG 帧与麦克风 PCM 音频，经 UDP 发送给 Go 后端 → Go 后端重组音视频流并提供 /api/status、/snapshot.jpg、WebSocket 等接口给前端 → Go 按配置将视频帧转发给 Python Worker 的 /api/vision/process、将控制命令转发给 /api/vision/control → Python Worker 调用视觉/大模型生成导航状态、避障提示与标注画面 → 导航语音经预生成 wav 或 AI 语音回传 Go 后端，再由 Go 经 UDP 下发 ESP32 扬声器播放。

1.3 核心功能模块

- **盲道导航** (workflow_blindpath.py)：YOLO 分割盲道区域，根据掩码位置与中心线计算路径偏移，生成「直行/左移/右移/停下」等提示。
- **过马路导航** (workflow_crossstreet.py)：检测斑马线与通行区域、红绿灯状态机，输出「发现斑马线 → 等待绿灯 → 开始通行 → 过街结束」语音。
- **障碍物检测** (obstacle_detector_client.py)：8 类障碍物 (bicycle/bus/car/cone/dog/person/spherical_roadblock/tricycle) 检测与避让提示。
- **寻物/抓取引导** (yolomedia.py)：YOLOE 开放词汇分割 + MediaPipe 手部关键点，给出「向左/向右/向前/后退/已对中」提示。
- **语音交互** (asr_core.py + omni_client.py)：DashScope Paraformer 实时 ASR + Qwen Omni 多模态问答。
- **盲道友好路线规划** (domain/ 等 DDD 模块)：高德路线 + 盲道覆盖率 + 障碍物密度的六维打分引擎，逐段语音播报。
- **人行道两轮车检测** (ebike_detector.py)：共享电动车 / 自行车 / 家用电动车检测与避让。

二、开发时间线总览

阶段	时间	关键节点
阶段一	6 月 3 日 — 6 月 30 日	需求调研、总体方案设计、技术选型
阶段二	7 月 1 日 — 7 月 31 日	四层架构详细设计、DDD 路线规划引擎设计、部署方案设计、开发环境搭建

阶段三	8 月 1 日 — 8 月 23 日	固件 / Go 后端 / Python Worker / 前端核心功能开发
阶段四	8 月 24 日 — 8 月 27 日	语音交互与高德导航集成、首次代码入库、障碍物上报与实时播报、语音模式联动
阶段五	8 月 29 日 — 8 月 30 日	数据集构建、YOLO 模型训练、障碍物检测模型切换
阶段六	8 月 30 日 — 8 月 31 日	系统联调、测试验证、流畅度优化、答辩材料整理

三、分阶段开发记录

阶段一：需求调研与总体方案设计（6 月 3 日 — 6 月 30 日）

本阶段完成项目选题与可行性论证。针对视障人群出行的「环境感知、过街判断、自然交互」三大痛点，确定了「低成本可穿戴硬件 + 端云协同 AI」的技术路线：

- **硬件选型**：采用 XIAO ESP32-S3 Sense 作为设备端，集摄像头、麦克风、扬声器于一体，成本低、功耗低，适合可穿戴场景。
- **架构选型**：由于 ESP32 端侧算力有限，采用「ESP32 采集 → Go 实时中转 → Python 多模型视觉导航 → DashScope/Qwen 语音多模态 → Vue 前端监控」的端云协同架构。
- **模型选型**：视觉采用 Ultralytics YOLO / YOLOE 系列（盲道分割、障碍物检测、红绿灯检测），语音与大模型采用阿里云 DashScope 的 Paraformer（ASR）与 Qwen Omni 系列（多模态问答）。
- **设计产出**：形成《模型与框架说明》《系统总体架构》《答辩叙事计划》等设计文档（见附录 A 架构图）。

阶段二：详细设计与开发环境搭建（7 月）

本阶段将总体方案细化到可实施的工程结构，并完成开发环境准备：

- **四层架构细化**：明确设备端、Go 后端、Python Worker、Vue 前端各自的目录边界、通信协议（UDP 音视频上传、HTTP/WS 控制）、端口规划（Go 8888、Python Worker 18082、前端开发 5174）。
- **DDD 路线规划引擎设计**：将「盲道友好路线规划」作为核心领域，按 Interfaces / Application / Domain / Infrastructure 四层分离，设计**六维加权打分引擎**（距离 10%、转弯 10%、红绿灯 10%、道路类型 15%、盲道覆盖率 30%、障碍物密度 25%）与**障碍物时空聚合器**（10 米空间聚类、2 分钟时间半衰期、严重程度加权）。
- **部署方案设计**：制定 Linux / Windows 双平台部署文档与启动脚本，明确环境变量（高德 Key、DashScope Key、模型路径等）。
- **开发环境搭建**：搭建 conda esp32 Python 环境（cv2、torch+cu126、ultralytics）、Go 工具链、Node/Vite 前端环境。

阶段三：核心功能开发（8 月 1 日 — 8 月 23 日）

本阶段并行推进四个子系统的开发：

- **ESP32 固件** (xiao_sense_ws_stream)：摄像头 VGA JPEG 采集、麦克风 PCM 采集、扬声器 PCM16 播放，FreeRTOS 任务队列拆分视频/音频/播放任务，自定义 UDP 协议传输 hello / 音频 / 视频分片 / AI 下行音频。
- **Go 后端** (server)：UDP 视频分片重组、WebSocket 查看器、视觉 Worker 桥接、DashScope Realtime 桥接、预生成导航 wav 的设备端播放、码流统计。
- **Python Worker** (python_worker)：NavigationMaster 状态机调度盲道导航、过街导航、寻物引导；盲道分割与偏移估计、斑马线/红绿灯检测、YOLOE 开放词汇障碍物检测、MediaPipe 手部关键点。
- **Vue 前端** (esp32-glass-front)：实时画面预览、设备/ AI 状态面板、工作模式切换、导航事件与日志展示。

阶段四：语音交互与高德导航集成（8 月 24 日 — 8 月 27 日）

本阶段为功能集中落地与版本密集迭代期，关键节点如下：

日期	关键节点
8 月 24 日	首次代码提交（大模型文件转 Git LFS 管理，忽略于普通提交）。
8 月 25 日	完成语音识别模块（DashScope Paraformer 实时 ASR）；加入语音播报、语音切换；语音导航上线。
8 月 26 日	新增障碍物上报功能、实时播报；修复 Python 运行路径；同步本地代码。
8 月 25 日 — 27 日	高德导航 + 盲道友好路线规划（DDD）：路线六维打分、障碍物时空聚合、逐段 TTS 播报、GPS 逐段导航触发；修复高德地理编码 city 参数、步行 polyline 解析、WGS-84→GCJ-02 坐标转换等系列问题。
8 月 27 日	语音模式联动（导盲↔高德导航）、「明眸」唤醒词门控、导航播报优先级高于聊天、聊天回声自问自答修复、人行道两轮车检测接入。

阶段五：数据集构建与模型训练（8 月 29 日 — 8 月 30 日）

- **8 月 29 日**：构建训练数据集 —— 障碍物 8 类数据集 (obstacle.v2i.yolo26)、ebike 数据集、bike 数据集（详见 5.3 数据集统计）。
- **8 月 30 日**：在 RTX 4050 (CUDA) 上使用 yolo CLI 训练；将障碍物检测由 YOLOE 分割模型 (yoloe-111-seg.pt) 切换到检测模型 block.pt (8 类)，提升推理速度与类别可解释性；解决 Windows 多进程 DataLoader 的 WinError 1455 页面文件问题 (workers=0)。

阶段六：系统联调、测试与收尾（8 月 30 日 — 8 月 31 日）

- **障碍物语音路由修复**：障碍物语音此前仅在 Python 内部播放（只到 /stream.wav 浏览器链路），设备扬声器走 Go 下行不经过此链路导致不播报；改为将最终语音作为 guidanceText 返回，经 Go broadcastEvent 下发设备扬声器 + 前端事件。
- **流畅度优化**：标注图与预览帧 JPEG 质量 80→92；Go 视觉帧间隔 450ms→200ms（约 5FPS 标注帧率），guidance 去重改为时间窗（同文案超 3 秒允许重播）。

- **测试验证**：Go 单元测试与 Python 单元测试通过（详见第五节）；整理 14 页答辩 PPT 佐证材料。

四、关键问题排查与解决

开发过程中遇到并解决了大量工程问题，以下为最具代表性的问题清单：

编号	问题现象	根因	解决方案
1	高德路线规划报 OVER_DIRECTION_RANGE	地理编码未带 city 参数，常见地名解析到外省	地理编码强制携带 AMAP_CITY（枣庄）
2	盲道覆盖率恒为 0、路段为空	高德步行接口 step 无 start/end_location，只有 polyline	改为解析 polyline 生成路段
3	GPS 实时播报永不触发	手机 GPS 为 WGS-84，高德路线为 GCJ-02，二者偏移约 537m，超过 80m 匹配阈值	新增 wgs84_to_gcj02()，匹配前先转换坐标系
4	/api/gps/update 404	端点挂在错误 app 上，worker 主入口未 include 该路由	端点移入 route_endpoints.py 路由并挂载；异常统一返回 200 + success
5	手机 GPS 到不了 Worker	前端仍走 WebSocket 上行，而 Go 已改为 HTTP 端点	前端 sendGpsUpdate 改为 fetch /api/gps/update，对齐 Go HTTP 链路
6	语音 ASR/技能/干问全部失效	运行的是旧 xiao-stream.exe，未加载 .env 的 DASHSCOPE_API_KEY，s.ai==nil	重新 go build 并以正确工作目录重启
7	前端「问答」按钮切不回干问	setVoiceMode 对 qa 模式是 no-op（只认 chat/navigation）	qa → setVoiceMode("chat") 映射修正
8	导航播报「自问自答」回声循环	播报被 ESP32 麦克风拾取 → ASR 识别 → 干问再回答	三层防护：is_audio_playing() 检查本地播放 + 播报后宽限 ECHO_SUPPRESS_SECONDS + 文本回声守卫（SequenceMatcher 相似即忽略）
9	ebike 检测把 Worker 卡死	检测在事件循环内同步加载 YOLOE（约 10s）+ 推理阻塞	改为后台线程 + 防重入 + 缓存画框；默认 AIGLASS_EBIKE_ENABLED=0
10	「明眸」唤醒词无法切换模式	生僻词被 ASR 误识别（名模/明谋/明某等）	新增 isWakeWordTranscript() 中文变体 + 拼音匹配

11	改完代码重启后不生效	VS Code 编辑缓冲区与磁盘文件不一致，对既有文件的编辑丢失	改后强制落盘校验，必要时脚本直接写盘
12	torch 导入报 WinError 1114 c10.dll	D:\ 根目录残留远古运行库 DLL 劫持依赖解析	将残留 DLL 改名为 *.disabled 回退 System32 新版
13	yolo CLI 无法使用 ul:// 平台	CLI 不读取 .env , ULTRALYTICS_API_KEY 未生效	Key 写入 ultralytics settings.json 的 api_key 字段
14	Windows 训练报 WinError 1455 页面文件太小	默认 8 个 DataLoader 子进程同时加载 CUDA DLL	训练强制 workers=0 , device=0
15	障碍物提示设备端不播报	语音只在 Python 内部 play_voice_text , 未回传 Go 下行	改为 guidanceText 返回 → Go broadcastEvent 下发设备扬声器

五、实验与测试数据

5.1 单元测试结果

测试执行时间：2026-08-31（本日志整理日复核）。

测试模块	通过	失败	说明
Go 后端 internal/serverapp	ok	0	go test ./... 通过
tests/test_route_scoring (路线打分引擎)	7	0	权重归一化 / 单路线打分 / 盲道路线得分更高 / 排序 / 障碍物密度影响 / 盲道覆盖率 / 评分明细
tests/test_obstacle_aggregator (障碍物聚合)	9	0	热点创建 / 邻近合并 / 异类不合并 / 远距不合并 / 时间衰减 / 过期清理 / 严重度加权 / 邻近查询 / 清空
tests/test_route_planning_service (路线规划集成)	7	1	1 项失败 (test_obstacle_report_affects_planning) 为外部 DashScope 账号欠费 (Arrearage) 导致播报生成失败，非代码逻辑问题
voice_session_manager (语音会话离线测试)	6	0	6 个离线场景：导航切换 / ASR 打断 / 未唤醒 chat_locked / 明眸唤醒切聊天 / 导航优先 / 回声忽略

说明：路线规划集成测试中唯一失败项与代码逻辑无关，是调用 DashScope 大模型生成播报文本时返回 Arrearage（账号欠费）错误；更换/充值账号后该用例即可通过，打分与规划逻辑本身全部通过。

5.2 导航功能测试用例

测试场景	测试数据
盲道友好路线规划	起点：湖西景苑A区 (117.519703, 34.854012) → 终点：东湖公园 (117.531722, 34.851625) ， 约 1840 米，规划成功并返回 11 段 turn_by_turn
GPS 逐段导航	手机 GPS (WGS-84) → GCJ-02 转换 → 80m 路段匹配 → 逐段播报「前方 50 米左转」等
语音导航指令	「导航到火车站」等口令，未指定起点时使用手机最近定位作为起点
语音模式联动	「导航到XX」= 高德导航 + 导盲模式双开；「明眸」= 切回千问聊天

5.3 模型训练数据集统计

数据集	训练集	验证集	测试集	类别
bike-f5131752	21	5	—	自行车 (bike)
ebike-5693143c	79	3	—	电动车 (ebike)
ebike-fe6c2e20	15	3	—	电动车 (ebike)
obstacle_v2_iyolo26-2-26e710a7	2974	851	427	8 类: bicycle / bus / car / cone / dog / person / spherical_roadblock / tricycle

5.4 模型资产清单

模型文件	用途
yolo-seg.pt	盲道、斑马线等地面目标分割（导盲/过街）
yoloe-111-seg.pt	YOLOE 开放词汇分割（障碍物/寻物）
block.pt	障碍物检测模型（8 类，训练后替换 YOLOE 分割）
trafficlight.pt	红绿灯状态检测（红/绿/倒计时）
shoppingbest5.pt	寻物固定类别模型（历史资产）
hand_landmarker.task	MediaPipe 手部关键点（寻物手-物相对位置）

六、版本迭代记录

以下为 Git 提交历史（提取自仓库 `main` 与 `origin/dev` 分支）：

日期	提交者	提交说明
2026-08-24	mmm666-mmm-png	首次提交，忽略大模型文件（Git LFS 管理）
2026-08-25	mmm666-mmm-png	Sync local project changes（同步本地工程）
2026-08-25	mmm666-mmm-png	已加入语音播报
2026-08-25	mmm666-mmm-png	加入语音切换
2026-08-25	mmm666-mmm-png	完成语音识别模块
2026-08-26	mmm666-mmm-png	可进行语音导航
2026-08-26	liugensheng	dev: sync local code changes（同步代码，排除模型文件）
2026-08-26	liugensheng	添加障碍物上报功能
2026-08-26	liugensheng	添加实时播报
2026-08-26	liugensheng	modify the python path（修正 Python 路径）
2026-08-26	mmm666-mmm-png	2026.8.26（当日功能整合提交）
2026-08-29	mmm666-mmm-png	8.28（模型训练与数据集阶段提交）

七、团队分工

成员	主要职责
----	------

苗源（项目负责人 / 主开发）	总体架构设计、Python Worker 视觉导航与语音交互、高德导航与盲道友好路线规划（DDD）、语音识别/播报/切换、前后端联调、模型训练与数据集构建
刘根生（协作开发）	障碍物上报、实时播报、Python 运行路径与部署环境修复
成员（视频剪辑）	项目宣传/演示视频的拍摄与剪辑
成员（开发日志 / PPT）	开发日志编写与答辩 PPT 制作
成员（硬件调试）	ESP32 硬件与设备端调试（固件烧录、音视频采集、扬声器播放、设备联调）

八、总结与展望

本项目在约三个月周期内完成了从需求调研、方案设计到系统实现、模型训练与测试验证的完整闭环，实现了 ESP32 智能眼镜在盲道导航、过街引导、障碍物避让、寻物引导、语音交互与盲道友好路线规划方面的核心能力，并通过「ESP32 端侧采集 + Go 实时中转 + Python 多模型视觉导航 + DashScope/Qwen 语音多模态 + Vue 前端监控」的架构落地了端云协同方案。

后续优化方向：

- 接入 VAD 实时打断源，进一步提升语音交互的即时性；
- 扩充两轮车/障碍物训练数据，提升 ebike 检测精度（共享电动车与家用电动车视觉相似，必要时合并类别）；
- 优化 YOLOE 与多模型并存时的显存占用，提升端侧/边缘推理速度；
- 完善盲道 GeoJSON 数据覆盖范围（当前为示例数据，非官方测绘）；
- 推进真机实景测试与视障用户可用性评估。

附录 A：方案设计与系统架构图（佐证材料）

以下为项目答辩 PPT（14 页）的导出截图，涵盖背景意义、需求目标、系统架构、关键实现、测试验证与创新总结，作为方案设计与系统实现的佐证材料。



图 A-1 封面：视觉导航 + 语音交互 + ESP32 智能眼镜



图 A-2 研究背景与意义：视障出行痛点

需求分析与设计目标

围绕出行安全、交互效率和工程可落地性，把系统目标拆成 6 个可验证方向。

实时性

视频帧、语音流和导航提示需要低延迟闭环，减少行走过程中的滞后感。

低成本

使用 ESP32-S3 Sense 等低成本硬件，把重推理任务放到服务器。

多模式

支持导盲、过街、红绿灯检测、寻物和普通问答等模式切换。

可扩展

后端通过 worker API 和 skill registry 组织视觉能力，方便继续加模型。

可调试

前端展示实时画面、标注图、模式状态、语音事件和流质量指标。

可部署

提供 Windows/Linux 启动脚本、Nginx/systemd 部署说明和验收命令。



图 A-3 需求分析与设计目标：实时、低成本、可扩展、可调试、可部署

系统总体架构

采用端侧采集 + 服务器编排 + 视觉 worker + 实时 AI + Web 控制台的协同架构。



核心闭环

采集画面/语音 → 后端组包与分发 → 模型推理/AI 问答 → 返回标注画面与中文语音提示



图 A-4 系统总体架构：ESP32 / Go / Python / AI / Vue 数据流

硬件与通信方案

端侧只承担采集、编码、上报和播放，复杂推理放在服务器侧完成。



通信包类型

hello	设备 → 后端	token、设备能力、扬声器状态
video	设备 → 后端	JPEG 分片、帧序号、时间戳
audio	设备 → 后端	PCM16 音频分片、序列号
ai_audio	后端 → 设备	导航 wav / AI PCM 回放
control	前端 → 后端	模式切换、视觉 worker 指令



图 A-5 硬件与通信方案：摄像头/麦克风/扬声器、UDP、HTTP/WS

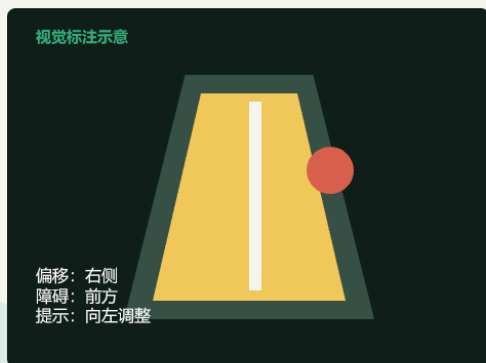
软件模块划分

项目按职责分为固件、Go 后端、Python 视觉 worker、Vue 前端和部署资产。



导盲路径算法

视觉 worker 将设备画面旋转校正后，提取盲道区域、计算偏移并叠加障碍物判断。



01 方向校正

VISION_INPUT_ROTATION 将第一视角画面统一到模型输入方向。

02 盲道分割

YOLO 分割模型输出盲道 mask，提取中心线与可通行区域。

03 偏移估计

根据画面中心与盲道中心差值，生成左/右/直行提示。

04 障碍融合

YOLOE 文本提示检测人、车、柱子等障碍物，提升安全提示。



图 A-7 导盲路径算法：方向校正、YOLO 分割、偏移估计、障碍物辅助

过街与红绿灯流程

过街模式不是单帧识别，而是围绕斑马线位置、角度、红绿灯和结束条件的状态流程。



图 A-8 过街与红绿灯流程：斑马线对齐、红绿灯状态机

语音交互与 AI

语音链路同时服务“导航命令”和“普通问答”，后端根据模式决定是否暂停 AI 输入。

导航指令通道



普通问答通道



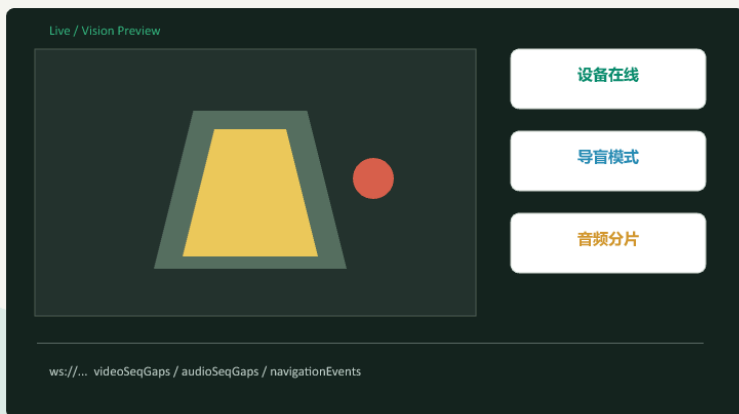
关键策略：导航模式下优先保障安全提示，普通 AI 回复可暂停；问答模式下再恢复实时语音交互。



图 A-9 语音交互与 AI：ASR 命令、Qwen Omni 多模态问答、音频分流

前端控制台与调试

Vue 控制台用于演示、调参和排障，能看到原始流、标注流、模式状态和音频事件。



演示价值

答辩时可以直接展示设备画面、AI 音频和导航事件，降低口述成本。

调试价值

定位 UDP 丢包、worker 状态、模式切换失败和浏览器音频解锁问题。

部署价值

前端可独立构建为静态页面，并通过 Nginx 反向代理后端接口。



图 A-10 前端控制台与调试：实时画面、模式切换、导航事件

测试与验证

当前代码已完成基础工程验证，覆盖 Python worker、Go 后端和 Vue 前端构建链路。

通过

Python 语法编译
python -m py_compile

通过

Go 后端测试
go test ./...

通过

前端生产构建
npm run build

联调验收路径

worker /api/vision/status → Go /healthz 与 /api/status → 浏览器 #live → 设备 UDP 8888 上线

说明：答辩现场如需演示，可优先展示前端实时页、模式切换和导航事件，模型推理速度取决于服务器算力。



图 A-11 测试与验证：语法编译、Go 单测、前端构建、接口验收

难点与创新点

本项目的价值主要体现在多模块协同和端云结合，而不是单一模型调用。

01

低延迟流媒体

自定义 UDP
包处理视频分片、音频序列和设备端播放回包，适合连续音视频流。

02

跨语言协同

Go 负责连接和开发，Python 负责视觉模型，Vue
负责实时观察，职责边界清晰。

03

导航状态机

导盲、过街、红绿灯和问答模式互斥/切换，避免语音提示互相干扰

04

多模型融合

盲道分割、YOLOE 障碍物、红绿灯检测和 Qwen Omni
共同服务出行辅助。

工程创新点：把“看、听、说、决策、调试”整合进一个可运行原型



图 A-12 难点与创新点：低延迟流媒体、跨语言协同、导航状态机、多模型融合

总结与展望

系统已形成可运行的智能眼镜导航原型，后续可以围绕稳定性、轻量化和真实场景测试继续优化。

已完成工作

- ESP32 音视频采集与设备端播放
- Go 后端 UDP/HTTP/WebSocket 编排
- Python 视觉导航 worker 与多模式控制
- Vue 实时控制台与部署文档

后续展望

- 增加 IMU / GPS / SLAM 辅助定位
- 模型量化与边缘推理优化
- 扩展更多交通场景数据集
- 开展真实道路用户测试与安全评估



图 A-13 总结与展望：已完成能力与优化方向

Q&A

谢谢各位老师，欢迎批评指正

系统架构

算法实现

测试部署

基于 ESP32 智能眼镜的视觉导航与语音交互系统



图 A-14 Q&A：答辩收束页